

Full-Stack Learning Guide

What each topic is, why it matters, and how it actually works — React & TypeScript · Python · C# & ASP.NET Core · SQL & MongoDB

Every lesson from the syllabus, walked through objective by objective with a full worked example — written so you can take your own notes straight from it.

Contents

MODULE A · REACT

- Lesson 1 — Introduction to React and JSX
- Lesson 2 — State Management with useState & useEffect
- Lesson 3 — React Router and Navigation
- Lesson 4 — Data Fetching and Context API

MODULE B · TYPESCRIPT

- Lesson 5 — TypeScript Basics and Setup
- Lesson 6 — TypeScript with React

MODULE A · REACT

- Lesson 7 — Advanced React — Custom Hooks & Performance

MODULE C · PYTHON

- Lesson 8 — Python OOP — Classes, Inheritance & Encapsulation
- Lesson 9 — Python File Handling & Data Processing
- Lesson 10 — Python Web APIs with Flask

MODULE D · C# / .NET

- Lesson 11 — C# Fundamentals & Object-Oriented Programming
- Lesson 12 — C# Control Flow & the Four Pillars of OOP
- Lesson 13 — Applying OOP — the Student Management System pattern
- Lesson 14 — Advanced C# — LINQ, Collections & Exception Handling

MODULE E · ASP.NET CORE

- Lesson 15 — Introduction to .NET & ASP.NET Core
- Lesson 16 — ASP.NET Core & Web API Development
- Lesson 17 — Entity Framework Core & Database Integration
- Lesson 18 — API Authentication & Security with JWT
- Lesson 19 — API Testing, Validation & Documentation with Swagger

MODULE F · DATABASES

- Lesson 20 — SQL Server — Queries, Joins & Database Design
- Lesson 21 — NoSQL with MongoDB & Deployment

Lesson 1: Introduction to React and JSX

What it is. React is a JavaScript library for building user interfaces out of small, reusable pieces called components. Instead of writing step-by-step code that finds a DOM element and mutates it, you describe what the UI should look like for the current data, and React takes care of turning that description into real changes on the page.

Why it matters. React is the most in-demand frontend skill in South Africa and globally — over 40% of local frontend job postings name it explicitly, and it's what companies like Facebook, Netflix and Airbnb build their products on. It's the difference between knowing JavaScript and being able to build the kind of interface companies actually hire people to ship.

How it works

Why React beats manual DOM manipulation. In vanilla JS, keeping the page in sync with changing data means manually finding elements and mutating them every time something changes (`document.getElementById(...).textContent = ...`), which gets fragile fast as an app grows. React flips this around: you write a function describing what the UI should look like for the current state, and React re-runs it and patches the DOM for you whenever that state changes.

Setting up a project with Vite. Vite is the modern build tool used to start a React project: `npm create vite@latest my-app -- --template react` scaffolds the files, `npm install` pulls dependencies, and `npm run dev` starts a dev server with near-instant reloads — much faster to work with than older tools.

JSX vs HTML. JSX looks like HTML but is really JavaScript: each tag compiles to a `React.createElement(...)` call. That's why its rules differ from HTML — `class` becomes `className` (`class` is a reserved JS word), event handlers are camelCase (`onClick`), and `{}` lets you drop real JavaScript expressions straight into the markup.

Functional components. A component is just a function, named in PascalCase, that returns JSX. React treats a lowercase tag as a built-in HTML element and a capitalized one as your own component — which is exactly why the naming convention isn't optional, it changes what React thinks it's rendering.

Props: passing data down. Props are how a parent hands data to a child, written as attributes: `<Greeting name="Thabo" />`. Inside the component they arrive as one object argument, and they are read-only — a component never modifies its own props.

Composition and reusability. Rather than one giant component, you build a UI from small, focused components nested inside each other. A `StudentCard` made of an `Avatar` and a `Bio` component is more reusable and easier to reason about than one component doing everything at once.

The virtual DOM. React keeps an in-memory copy of the UI tree. When state changes, it builds a new version of that tree, diffs it against the previous one (reconciliation), and applies only the specific real-DOM changes needed — instead of re-rendering the whole page from scratch.

Practical example

Composing a `StudentCard` out of smaller components

```

function Avatar({ src, alt }) {
  return <img className="avatar" src={src} alt={alt} />;
}

function StudentCard({ name, course }) {
  return (
    <div className="card">
      <Avatar src="/avatars/default.png" alt={name} />
      <h3>{name}</h3>
      <p>Enrolled in: {course}</p>
    </div>
  );
}

// usage
<StudentCard name="Naledi Mokoena" course="Full-Stack Development" />

```

Walking through it: Avatar and StudentCard are both plain functions returning JSX. Avatar takes src and alt as props and renders an image. StudentCard takes name and course as props, and composes Avatar inside itself alongside plain JSX elements — that's composition. When <StudentCard ... /> is rendered with real values, React calls the function, gets back a JSX tree, and turns it into actual DOM nodes on the page.

Lesson 2: State Management with `useState` & `useEffect`

What it is. Hooks are functions that let a plain function component hold state and run side effects. `useState` gives a component memory that survives between renders; `useEffect` lets it react to render or to changing values by running code outside the normal render flow — fetching data, setting up a subscription, syncing with something outside React.

Why it matters. State is what turns a static page into an application. Every interactive feature — a form, a counter, a dashboard that updates live — needs state managed correctly, which is why this is treated as the single most fundamental skill for any frontend role.

How it works

The `useState` hook. `const [count, setCount] = useState(0)` gives you the current value and a setter function. Calling the setter schedules a re-render with the new value — you never mutate the state variable directly (no `count++`).

Handling events. Event props like `onClick`, `onChange` and `onSubmit` take a function reference, called by React when the event fires: `<button onClick={handleClick}>`, not `onClick={handleClick()}`, which would call it immediately during render.

Updating state from events. Inside a handler you read the event or current state and call the setter with a new value: for state that depends on the previous value, pass a function — `setCount(c => c + 1)` — so React always works from the latest value, even if several updates are batched together.

Managing multiple state variables. Prefer several focused `useState` calls (name, email, errors) over one large state object, unless the values genuinely change together — it keeps each update simple and avoids accidentally overwriting unrelated fields.

`useEffect` and the dependency array. `useEffect(fn, deps)` runs `fn` after the DOM commits. An empty array means 'run once on mount'; a list of values means 're-run whenever any of these change'; no array at all means 'run after every render'.

Cleanup and the render cycle. Returning a function from the effect lets you cancel a timer, remove a listener, or abort a fetch before the effect re-runs or the component unmounts — without it, old subscriptions can pile up every time the effect re-fires.

Controlled components and forms. An input is 'controlled' when its value comes from state and `onChange` writes back to that state on every keystroke — React owns the source of truth instead of the DOM, which is what lets you validate, transform, or reset the value programmatically.

Practical example

A controlled form field with a live character counter

```
function BioField() {
  const [bio, setBio] = useState("");

  useEffect(() => {
    document.title = `Bio (${bio.length}/200)`;
  }, [bio]);

  return (
    <textarea
      value={bio}
      maxLength={200}
      onChange={(e) => setBio(e.target.value)}
    />
  );
}
```

Walking through it: bio is state, starting as an empty string. The textarea's value is tied to bio (controlled component), and onChange calls setBio with whatever the user just typed, which triggers a re-render showing the new text. The useEffect watches bio specifically ([bio]), so every time it changes, the tab title updates to show the live character count — a small, realistic side effect kept out of the render logic itself.

Lesson 3: React Router and Navigation

What it is. React Router lets a single-page React app behave like a multi-page site: different URLs render different components, links update the URL without a full page reload, and the browser's back/forward buttons still work as expected.

Why it matters. Nearly every real application — e-commerce sites, dashboards, content platforms — needs more than one view, and most React job postings explicitly expect Router experience. Without it, you can only ever build a single-screen app.

How it works

Installing and setting up routes. After installing `react-router-dom`, wrap the app in `<BrowserRouter>` and declare each view as a `<Route path="..." element={<Page />} />` inside `<Routes>` — React Router matches the current URL against these and renders the matching one.

Navigating with `Link` and `NavLink`. `<Link to="/students">` renders an anchor tag but intercepts the click to update the URL client-side, with no full reload; `NavLink` does the same but also exposes whether it's the currently active route, for styling a nav menu.

Dynamic routes and URL parameters. A route declared as `/students/:id` makes the `id` segment available inside the matched component via `useParams()`, which returns an object like `{ id: '42' }` parsed straight from the URL.

Handling 404s and redirects. A route with `path="*"`, placed last in the list, catches any URL that didn't match an earlier, more specific route — that's your 404 page. A `<Navigate to="/login" />` element redirects declaratively.

Nested routes. A parent route's element can include an `<Outlet />`, which is where its matched child route renders — useful for a shared layout (like a sidebar) that wraps several different pages.

Programmatic navigation with `useNavigate`. `const navigate = useNavigate()` gives you a function to redirect from code — for instance, after a form submits successfully: `navigate('/students')`.

Passing state between routes. `navigate('/students/42', { state: { from: 'search' } })` carries extra data to the next route without putting it in the URL, readable on the other side via `useLocation().state`.

Practical example

A student detail page reached via a dynamic route

```

<Routes>
  <Route path="/students" element={<StudentList />} />
  <Route path="/students/:id" element={<StudentDetail />} />
  <Route path="*" element={<NotFound />} />
</Routes>

function StudentDetail() {
  const { id } = useParams();
  const navigate = useNavigate();

  return (
    <div>
      <p>Showing student {id}</p>
      <button onClick={() => navigate(-1)}>Back</button>
    </div>
  );
}

```

Walking through it: Visiting `/students/42` matches the second route, so React Router renders `StudentDetail` and makes '42' available through `useParams().id`. The Back button uses `useNavigate()` to go back one step in history programmatically, without needing a `<Link>`. Any URL that doesn't match either `/students` or `/students/:id` — like a typo — falls through to the catch-all `NotFound` route at the bottom.

Lesson 4: Data Fetching and Context API

What it is. This lesson connects React to the outside world in two directions: pulling data from a backend API with `useEffect` and `fetch`, and sharing state across many components with the Context API so it doesn't have to be threaded through every layer as props.

Why it matters. Almost every real application displays data that comes from a server — a news feed, a product catalog, a dashboard — and needs some state (like the logged-in user) available in many places at once. This is the bridge between frontend and backend work, and core to any full-stack role.

How it works

Fetching data in `useEffect`. The request is kicked off after the component renders, inside a `useEffect` with an empty (or param-based) dependency array, since `fetch` is a side effect and shouldn't run directly during render.

Handling loading, success and error states. Track loading, data and error as separate state values: loading starts `true` and flips to `false` once the request settles either way, data holds a successful result, error holds anything that went wrong — so the UI can render the right thing at each stage instead of guessing from a single value.

Displaying fetched data. Once data arrives, render it the normal React way — map over an array to a list of components, and guard against data still being null while loading is `true`.

The Context API. `createContext(default)` creates a context object; wrapping part of the tree in `<MyContext.Provider value={...}>` makes that value available to every descendant, read with `useContext(MyContext)`.

Avoiding prop drilling. Without Context, sharing a value like the current user with a deeply nested component means passing it as a prop through every component in between, even ones that never use it themselves — Context lets any descendant read the value directly.

Combining fetching with routing. A route like `/students/:id` commonly triggers a fetch keyed on `useParams().id` inside `useEffect`, re-fetching automatically whenever the id in the URL changes, since id would be listed in the dependency array.

Cleanup and dependency arrays. If a component can unmount before a fetch resolves, guard the state update (a boolean flag set in the cleanup function, or an `AbortController`) so you don't call `setState` on an unmounted component; always double-check the dependency array matches what the effect actually reads, or it will re-run too often or too rarely.

Practical example

Fetching a student list and sharing the logged-in user via Context

```

const UserContext = createContext(null);

function StudentList() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("/api/students")
      .then((res) => res.json())
      .then(setStudents)
      .catch(setError)
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Could not load students.</p>;
  return <ul>{students.map((s) => <li key={s.id}>{s.name}</li>)}</ul>;
}

```

Walking through it: On mount, the effect fires once (empty array), calls the API, and funnels the result into either students (success) or error (failure), always turning loading back off in finally. The render function checks loading and error first, so the list only renders once real data is actually available — a `UserContext.Provider` higher up the tree would let any component here read the logged-in user with `useContext(UserContext)` without it being passed down as a prop.

Lesson 5: TypeScript Basics and Setup

What it is. TypeScript is JavaScript with an optional static type layer checked at compile time. You describe the shape of your data and functions, and the compiler catches a whole category of mistakes — wrong argument, missing field, typo'd property — before the code ever runs.

Why it matters. TypeScript is being adopted rapidly by South African and global companies (Microsoft, Google, Airbnb, Slack) because it reduces bugs, makes refactoring safer, and gives much better autocomplete. Many React roles now list it as a requirement, not a nice-to-have.

How it works

Setting up a project. `tsconfig.json` configures the compiler — strictness, target JS version, module resolution. Most modern setups (Vite included) generate a sensible one automatically when you scaffold a TypeScript project.

Basic types. `string`, `number`, `boolean`, array types (`string[]`), tuples (a fixed-length array where each position has its own type), and `enum` for a fixed set of named constants.

any vs unknown. `any` switches off type checking entirely for that value; `unknown` is the safe alternative — it forces you to narrow the value (e.g. with `typeof`) before you're allowed to use it.

interfaces and type aliases. Both describe an object's shape. Interfaces can be extended and re-opened (declaration merging), which reads naturally for public API shapes; type aliases are required when you need a union (`"admin" | "student"`) or an intersection type.

Union and intersection types. A union (`A | B`) means 'either shape is valid'; an intersection (`A & B`) means 'must satisfy both shapes at once' — useful for combining smaller, focused types into one.

Type inference vs explicit types. TypeScript infers types automatically wherever it reasonably can (`let age = 20` is inferred as `number`) — you mainly need explicit annotations on function signatures, exported values, and cases the compiler can't infer on its own, like an empty array or a null-starting value.

Generics. `<T>` lets a function or type work with whatever type is passed in while keeping full type checking, instead of writing one version per type or falling back to `any`.

Practical example

A typed student record and a generic helper

```
interface Student {
  id: number;
  name: string;
  status: "active" | "graduated"; // union type
}

function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
  return items.find((item) => item.id === id);
}

const match = findById<Student>(students, 42); // typed as Student | undefined
```

Walking through it: Student is an interface describing exactly what shape a student record has, including a union type restricting status to two literal strings. findById is generic: T is constrained to 'anything with a numeric id', so it works for students, courses, or anything else with that shape, and the caller still gets a properly-typed result back (Student | undefined here) instead of any.

Lesson 6: TypeScript with React

What it is. Typing a React component means typing three things precisely: what props it accepts, what shape its state holds, and what shape its events carry — so a mistake at the call site is caught by the compiler instead of showing up as a bug in the browser.

Why it matters. Combining React with TypeScript is becoming the industry standard for professional frontend work: it catches prop mistakes immediately, makes large refactors far safer, and is one of the most valued skill combinations in the South African job market and beyond.

How it works

Typing functional components. Define a props interface and destructure the function's argument against it: `function Card({ title }: CardProps)` — the compiler then checks every place `Card` is used against that shape.

Defining prop types. Use an interface (or type alias) listing each prop and its type, marking genuinely optional ones with `?:`: `role?: "admin" | "student"`.

Typing `useState` and `useEffect`. TypeScript infers `useState`'s type from its initial value; when that value doesn't convey the full type — starting at `null`, for example — annotate it explicitly: `useState<User | null>(null)`. `useEffect` itself needs no special typing beyond typing what it reads.

Typing events. React re-exports DOM event types parameterized by element:

`React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>` — matching these to the actual element keeps every property on the event correctly typed.

Typing children and composition. `children` is typed `React.ReactNode`, which covers strings, numbers, elements, fragments and arrays of any of those — the right type for 'anything React can render'.

Generic components. A component itself can take a type parameter, e.g. a reusable `List<T>` that renders any array with a supplied render function, keeping full type safety no matter what data type it's given.

React.FC: pros and cons. `React.FC` types a function as a component and used to add an implicit `children` prop automatically; many teams now type the function directly instead, since `React.FC` can fight with generic components and no longer adds `children` by default in recent versions.

Typing custom hooks. A custom hook's return type should be explicit or clearly inferable — for a tuple-style return like `useState`'s, use a tuple type `(: [User | null, (u: User) => void])` so destructuring keeps its order-based typing.

Practical example

A typed, reusable button and a typed custom hook

```
interface ButtonProps {  
  label: string;  
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;  
}  
  
function Button({ label, onClick }: ButtonProps) {  
  return <button onClick={onClick}>{label}</button>;  
}  
  
function useUser(): [User | null, (u: User) => void] {  
  const [user, setUser] = useState<User | null>(null);  
  return [user, setUser];  
}
```

Walking through it: ButtonProps pins down exactly what Button needs: a string label and a click handler typed against a real button's MouseEvent. useUser wraps a typed useState call and declares its own return type as a tuple, so callers destructuring it, like `const [user, setUser] = useUser()`, get the same type safety as calling useState directly.

Lesson 7: Advanced React — Custom Hooks & Performance

What it is. This lesson covers the techniques that keep a growing React app fast and its logic reusable: custom hooks package up repeated stateful logic, memoization avoids repeating expensive work, and code splitting keeps the initial bundle small.

Why it matters. Performance is a key differentiator for senior developers, and these patterns are what let production apps stay fast and responsive even with large datasets and complex UIs — the difference between an app that feels snappy and one that visibly lags as it grows.

How it works

Building a custom hook. A custom hook is a function whose name starts with 'use', calling other hooks internally to package up reusable stateful logic — for example a `useLocalStorage(key, initial)` hook that wraps `useState` plus a `useEffect` that syncs its value to `localStorage`.

React.memo for component memoization. Wrapping a component in `React.memo(MyComponent)` makes it skip re-rendering when its props are shallowly equal to last time — useful for a component that renders often with unchanged props.

useMemo for expensive calculations. `useMemo(() => computeExpensive(data), [data])` caches the *result* of a calculation between renders, recomputing only when a dependency actually changes — worth it specifically when the computation is genuinely expensive.

useCallback for stable function references. `useCallback(fn, deps)` caches the *function itself*, which matters when that function is passed to a memoized child (so it doesn't defeat `React.memo`) or used as another hook's dependency.

Code splitting and lazy loading. `const Settings = React.lazy(() => import('./Settings'))` loads that component's code only when it's actually rendered, wrapped in `<Suspense fallback={...}>` to show something while it loads — shrinking what has to download up front.

Profiling with React DevTools. The Profiler tab records which components rendered and why, surfacing unnecessary re-renders — the right first step before reaching for memoization, since optimizing without measuring often targets the wrong component.

Advanced component patterns. Compound components (like a `Tabs` component whose `Tab` children coordinate through shared context) and render props (a component that takes a function as a prop to decide what to render) are patterns for building flexible, reusable component APIs.

Practical example

A custom hook plus a memoized expensive filter

```

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

function StudentTable({ students, query }) {
  const filtered = useMemo(
    () => students.filter((s) => s.name.includes(query)),
    [students, query]
  );
  return <Table rows={filtered} />;
}

```

Walking through it: `useLocalStorage` behaves just like `useState` from the outside, but persists its value to `localStorage` automatically via its own internal effect — any component can now get persisted state with one line. In `StudentTable`, filtering a potentially large `students` array only needs to re-run when `students` or `query` actually change, not on every render of the parent — that's exactly the kind of genuinely-expensive-enough case `useMemo` is meant for.

Lesson 8: Python OOP — Classes, Inheritance & Encapsulation

What it is. Object-oriented Python groups related data and the behaviour that operates on it into a class, and lets new classes reuse and specialize existing ones through inheritance — the same foundation frameworks like Django and Flask are built on.

Why it matters. OOP is fundamental to professional software development and comes up constantly in technical interviews. Employers value developers who can design scalable, maintainable systems using these principles, not just write one-off scripts.

How it works

Classes, objects, and `__init__`. A class is a blueprint; calling it — `Student("Amara")` — creates an object. `__init__` is the constructor that runs automatically at creation time to set up the object's initial state, with `self` as the instance being built.

Instance, class, and static methods. An instance method takes `self` and operates on one object; a `@classmethod` takes `cls` and operates on the class itself (often used as an alternative constructor); a `@staticmethod` takes neither — it's just a function namespaced inside the class for organization.

Special (dunder) methods. `__str__` gives the readable, user-facing string (used by `print()`); `__repr__` gives the unambiguous, developer-facing one (used by the REPL/debugger, and the fallback for `__str__`); `__eq__` defines what `'=='` means between two instances.

Inheritance and method overriding. `class Dog(Animal):` makes `Dog` inherit everything `Animal` has; redefining a method with the same name overrides it, and `super().__init__(...)` calls the parent's constructor so its own setup still runs before `Dog` adds its own.

Polymorphism and duck typing. Python doesn't check an object's declared type before calling a method on it — if the object has the right method, the call works, regardless of its class. That's duck typing: 'if it walks like a duck and quacks like a duck...'

Encapsulation with naming conventions. A single leading underscore (`_balance`) signals 'internal, don't touch' purely by convention. A double leading underscore (`__balance`) triggers name mangling (stored as `_ClassName__balance`) — Python has no true enforced private access modifier.

Properties and decorators. `@property` lets a method be accessed like an attribute (`student.age` instead of `student.age()`), which is how you add validation or a computed value without breaking existing attribute-style access.

Composition vs inheritance. Inheritance models 'is-a' (a `Dog` is an `Animal`); composition models 'has-a' (a `Car` has an `Engine`, stored as an attribute). Composition is often the more flexible choice when the relationship isn't a strict specialization.

Practical example

A small class hierarchy with a computed property

```

class Animal:
    def __init__(self, name, age):
        self.name = name
        self._age = age

    @property
    def is_adult(self):
        return self._age >= 2

    def speak(self):
        return f"{self.name} makes a sound"

    def __str__(self):
        return f"Animal({self.name})"

class Dog(Animal):
    def speak(self): # override
        return f"{self.name} barks"

rex = Dog("Rex", 3)
print(rex.speak())    # polymorphism: same call, Dog's behaviour
print(rex.is_adult)   # property accessed like an attribute

```

Walking through it: Animal defines shared state (name, age) and behaviour (speak, __str__); Dog inherits all of it and overrides speak with its own version. Calling rex.speak() runs Dog's version, not Animal's — that's polymorphism in action. is_adult is defined with @property, so it's read without parentheses even though it's really a method doing a comparison.

Lesson 9: Python File Handling & Data Processing

What it is. This lesson covers the everyday shape of automation work in Python: reading data in from a file, transforming it, and writing it back out — reliably, across platforms, and without leaking resources or crashing on bad input.

Why it matters. These are immediately workplace-applicable skills — processing CSV reports, converting file formats, organizing files, generating automated reports — the kind of practical automation many South African companies need developers to handle as part of everyday backend work.

How it works

Context managers and the with statement. `with open(path) as f:` guarantees the file is closed when the block exits, whether it exits normally or via an exception — removing an entire class of resource-leak bugs compared to manual `open()/close()`.

Reading and writing text files. `f.read()` reads the whole file, for `line in f:` streams it line by line without loading it all into memory at once — the better choice for large files.

Working with CSV. `csv.DictReader(f)` turns each row into a dictionary keyed by the header row, which is usually easier to work with than raw lists of values; `csv.DictWriter` writes them back out the same way.

Parsing and generating JSON. `json.load(f)` parses a JSON file into Python dicts/lists; `json.dump(data, f)` writes them back out — the standard way to exchange structured data with an API.

Cross-platform paths with pathlib. `Path("data") / "students.csv"` builds a path that works correctly on both Windows and Unix, instead of manually joining strings with slashes or backslashes.

Exception handling for file operations. Wrap file operations in `try/except/finally`, catching specific exceptions like `FileNotFoundError` rather than a bare `except:`, so genuine bugs elsewhere in the block still surface instead of being silently swallowed.

File encodings. Passing `encoding="utf-8"` explicitly to `open()` avoids `UnicodeDecodeError` surprises caused by a platform's default encoding differing from the file's actual encoding — worth doing as a habit, not just when something breaks.

Building automation scripts. Combine these pieces into small, composable functions — one that reads, one that transforms, one that writes — so the script is easy to test and reuse for a different input file or output format later.

Practical example

Reading a CSV report and writing a filtered JSON summary

```

import csv, json
from pathlib import Path

def load_students(path):
    with open(path, encoding="utf-8") as f:
        return list(csv.DictReader(f))

def top_performers(students, threshold=70):
    return [s for s in students if float(s["average"]) >= threshold]

src = Path("data") / "students.csv"
try:
    students = load_students(src)
    report = top_performers(students)
    with open(Path("data") / "top_performers.json", "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)
except FileNotFoundError:
    print(f"No such file: {src}")

```

Walking through it: `load_students` opens the CSV with an explicit encoding inside a `with` block, and `DictReader` turns every row into a dict. `top_performers` is a small, separate function that filters that data — kept apart from the file I/O so it's easy to test on its own. The whole pipeline is wrapped in `try/except FileNotFoundError` specifically, so a missing input file prints a clear message instead of a raw traceback, while any other unexpected error would still surface normally.

Lesson 10: Python Web APIs with Flask

What it is. Flask is a lightweight Python web framework used to expose backend logic as an HTTP API — the layer that lets a frontend like React (or Postman, or another service) talk to Python code over the network using standard REST conventions.

Why it matters. This bridges frontend and backend skills into full-stack capability, and Flask is widely used in South African companies for microservices and backend services. Being able to build APIs in more than one language (here, alongside C#) makes a developer more versatile and employable.

How it works

Setting up a Flask project. A minimal app is `app = Flask(__name__)` plus route functions, run with `flask run` or `app.run(debug=True)` for local development with auto-reload.

Creating routes and HTTP methods. `@app.route("/students", methods=["GET", "POST"])` maps a URL and one or more HTTP methods to a Python function — GET for reading, POST for creating, PUT for updating, DELETE for removing.

Reading request data. `request.args` reads query-string parameters (`?page=2`), `request.json` parses a JSON request body, and `request.form` reads standard form-encoded POST data.

Returning JSON responses with status codes. `return jsonify(data), 201` returns a JSON body with an explicit status code — 200 for a successful read, 201 for created, 400 for invalid input, 404 for missing, 500 for a server-side failure.

Implementing CRUD operations. Each REST verb maps onto a data operation: GET `/students` lists, GET `/students/<id>` reads one, POST `/students` creates, PUT `/students/<id>` updates, DELETE `/students/<id>` removes — the URL names the resource, the verb names the action.

Handling errors gracefully. Check for missing/invalid input explicitly and return a 400 with a clear message rather than letting the request crash into a raw 500 — a client should always get a meaningful response, not a stack trace.

REST principles and conventions. URLs are plural resource nouns (`/students`, not `/getStudents`), and the same URL means different things depending purely on the HTTP verb used against it — that consistency is what makes an API predictable to use.

Testing APIs. Postman or Thunder Client let you send real requests with chosen methods, headers, and bodies to manually verify each endpoint behaves as expected before a frontend ever calls it.

Practical example

A minimal CRUD-style Students API

```

from flask import Flask, request, jsonify

app = Flask(__name__)
students = {}
next_id = 1

@app.route("/students", methods=["POST"])
def create_student():
    global next_id
    data = request.json
    if not data or "name" not in data:
        return jsonify({"error": "name is required"}), 400
    students[next_id] = {"id": next_id, "name": data["name"]}
    next_id += 1
    return jsonify(students[next_id - 1]), 201

@app.route("/students/<int:student_id>", methods=["GET"])
def get_student(student_id):
    student = students.get(student_id)
    if not student:
        return jsonify({"error": "not found"}), 404
    return jsonify(student), 200

```

Walking through it: `create_student` reads the JSON body, validates that 'name' is present (returning 400 with a clear message if not), then stores and returns the new record with a 201. `get_student` reads the id straight from the URL (Flask converts it to an int via `<int:student_id>`), and returns 404 with a message if that id doesn't exist — every path returns a status code that actually matches what happened.

Lesson 11: C# Fundamentals & Object-Oriented Programming

What it is. C# is a strongly-typed, object-oriented language from Microsoft and the foundation for everything later in the module — syntax, typing rules, and the OOP building blocks that ASP.NET Core and Entity Framework are built on top of.

Why it matters. C# is one of the most in-demand languages in South Africa, especially in banking, insurance and corporate sectors where .NET is the standard stack. It opens the door to well-paying corporate development roles.

How it works

Strong typing and data types. Every variable has a fixed, compiler-checked type: `int`, `string`, `bool`, `double`, `decimal` (for money, since it avoids float rounding errors), and arrays.

Control structures. `if/else` `if/else` and `switch` branch execution; `for`, `while`, and `foreach` (the natural choice for iterating a collection) `repeat` it.

Classes and encapsulation. A class groups data (fields/properties) with the methods that act on it. Properties (`public string Name { get; set; }`) expose fields with optional validation logic, rather than exposing raw fields directly.

Methods, properties, and constructors. A constructor runs when an object is created via `new Student( ... )`, typically used to require the data that makes the object valid from the start rather than leaving it settable piecemeal afterward.

Access modifiers. `public` is accessible from anywhere, `private` only inside the declaring class, and `protected` in the declaring class and its subclasses — unlike Python, these are compiler-enforced, not just convention.

Value types vs reference types. Value types (`int`, `struct`) are copied when assigned or passed to a method; reference types (`class`) are assigned/passed as a reference to the same object on the heap, so mutating it through one reference is visible through every other reference to it.

Namespaces and using directives. Namespaces group related types and avoid naming collisions across libraries; `using System.Collections.Generic;` at the top of a file lets you reference types in that namespace without writing it out every time.

Naming conventions. PascalCase for classes, methods, and public properties (`GetStudent`); camelCase for local variables and parameters (`studentId`) — consistently followed across the whole .NET ecosystem.

Practical example

A Student class demonstrating properties, a constructor, and encapsulation

```

public class Student
{
    public string Name { get; set; }
    public int Id { get; }           // read-only after construction
    private double _average;

    public Student(int id, string name)
    {
        Id = id;
        Name = name;
    }

    public void RecordAverage(double average)
    {
        if (average < 0 || average > 100)
            throw new ArgumentOutOfRangeException(nameof(average));
        _average = average;
    }

    public string Describe() => $"{Name} (avg: {_average})";
}

```

Walking through it: Id is exposed as read-only (no setter), so once a Student is constructed its identity can't be changed from outside — that's the constructor enforcing valid state. _average is a private field, only reachable through the public RecordAverage method, which validates the value before storing it — that validation is exactly what encapsulation buys you over a plain public field.

Lesson 12: C# Control Flow & the Four Pillars of OOP

What it is. Before advanced C# features, everything rests on controlling how a program executes and structuring code around objects. This lesson is the four pillars of OOP — encapsulation, inheritance, polymorphism, abstraction — plus the loop and branching constructs every method is built from.

Why it matters. Strong knowledge of control flow and OOP is highly valued by South African employers because it enables developers to write scalable, maintainable, industry-standard applications — and it's the foundation everything else in the .NET module (LINQ, Entity Framework, ASP.NET Core) builds on.

How it works

Conditional statements. `if / else if / else` for branching on arbitrary conditions; `switch` for branching cleanly on one value matching several possible cases.

Looping constructs. `for` when you know the number of iterations, `foreach` for iterating a collection, `while` when the loop depends on a condition checked up front, and `do-while` when the body must run at least once before that condition is checked.

Reusable methods. A method groups a chunk of logic behind a name, taking typed parameters and returning a typed result — the basic unit of reuse in C#, avoiding copy-pasted logic across the codebase.

Encapsulation. Bundling data with the methods that operate on it, and hiding internal details behind access modifiers/properties, so external code interacts through a controlled interface instead of reaching directly into an object's internals.

Inheritance. `class GraduateStudent : Student` lets a subclass reuse everything a base class already defines, adding or overriding only what's actually different.

Polymorphism. A base method marked `virtual` (or `abstract`) can be redefined in a subclass with `override`; calling it through a base-typed reference still dispatches to the subclass's version at runtime — that's what lets one line of calling code work correctly across many different subclasses.

Abstraction. An `abstract class` or `interface` exposes only what a consumer needs to know, hiding how it's actually implemented — consumers code against the abstraction, not the concrete details.

Method overloading vs overriding. Overloading is multiple methods with the same name but different parameter lists in the same class, resolved at compile time; overriding is a subclass redefining a `virtual/abstract` base method with the same signature, resolved at runtime.

Practical example

Encapsulation, inheritance, and polymorphism together

```

public abstract class Person
{
    public string Name { get; }
    protected Person(string name) { Name = name; }

    public abstract string Describe(); // abstraction: no implementation here
}

public class Student : Person
{
    public double Average { get; private set; }

    public Student(string name, double average) : base(name)
    {
        Average = average;
    }

    public override string Describe() => $"{Name}, average {Average}"; // polymorphism
}

Person p = new Student("Kabelo", 82.5);
Console.WriteLine(p.Describe()); // calls Student's version, even through a Person reference

```

Walking through it: Person is abstract and declares Describe() with no body, forcing every subclass to supply its own — abstraction. Student inherits Name and adds Average, with a private setter (encapsulation: only Student itself can change it). Even though p is declared as type Person, calling p.Describe() runs Student's override, because the base method was abstract — that's polymorphism resolving the actual call at runtime based on the real object type.

Lesson 13: Applying OOP — the Student Management System pattern

What it is. The bootcamp's recurring practical project is a console-based Student Management System that lets a user add, search, update, and display student records. It's worth understanding as a design pattern — interviewers often ask you to design something structurally identical.

Why it matters. Mastering this pattern is fundamental to professional C# development and lays the groundwork for everything that follows in the module — LINQ, Entity Framework, ASP.NET Core all reuse the same shape of 'a class holding data, a class holding operations on it'.

How it works

Modelling the domain. A Student class holds only data (name, id, grades); a separate manager or service class holds the operations on that data (add, search, update, list) — keeping 'what a student is' separate from 'what you can do with a list of students'.

Using constructors to enforce valid state. Requiring the required fields through the constructor, rather than leaving them settable after the fact, makes it much harder to end up with a half-built, invalid Student floating around the program.

Storing records in a collection. A console CRUD app typically holds records in a `List<Student>` in memory, standing in for a real database until Entity Framework is introduced later in the module.

Add, search, update, and display operations. Each menu action maps to one manager method: `Add(student)` appends to the list, `FindById(id)` searches it (typically with LINQ's `FirstOrDefault`), `Update(id, ...)` finds and modifies a record, and a display method prints every current record.

Applying decision-making and loops. The console menu itself is a loop (keep showing options until the user quits) wrapping a switch or if-chain that dispatches to the right manager method based on what the user typed — control flow and OOP working together, not in isolation.

Separating concerns. Input/output — reading console commands, printing results — stays separate from the logic that actually adds, searches, or updates records. That's the same controller/service split a real API's layers make later, just in console form.

Practical example

The manager/data split for a console Student Management System

```

public class Student
{
    public int Id { get; }
    public string Name { get; set; }
    public Student(int id, string name) { Id = id; Name = name; }
}

public class StudentManager
{
    private readonly List<Student> _students = new();

    public void Add(Student s) => _students.Add(s);

    public Student? FindById(int id) =>
        _students.FirstOrDefault(s => s.Id == id);

    public void Display() =>
        _students.ForEach(s => Console.WriteLine($"{s.Id}: {s.Name}"));
}

// in Main: read a menu choice, then call manager.Add(...) / manager.Display() etc.

```

Walking through it: Student is pure data with a constructor that requires an id and name up front. StudentManager owns the actual List<Student> as a private field, and every operation on that list — adding, searching, displaying — goes through one of its public methods. Main's job is reduced to reading what the user typed and calling the matching manager method, never touching _students directly.

Lesson 14: Advanced C# — LINQ, Collections & Exception Handling

What it is. These are the tools that make everyday C# concise instead of loop-heavy: LINQ for querying data in memory using declarative methods, the right generic collection for the access pattern you actually need, and structured, informative error handling.

Why it matters. These concepts are widely used in enterprise applications and are the direct foundation for working with Entity Framework and ASP.NET Core — LINQ especially shows up constantly in professional C# code and in interviews.

How it works

Querying with LINQ. `Where()` filters, `Select()` projects/transforms each item, `OrderBy()/OrderByDescending()` sorts, `GroupBy()` buckets items by a key, and `Join()` combines two sequences on a matching key — chained together like a pipeline.

Deferred execution. A LINQ query doesn't run when it's written — only when enumerated, via a `foreach` or a call like `.ToList()`. If the source collection changes in between, the query sees the changed data at that point, not a frozen snapshot.

Generic collections. `List<T>` is the general-purpose default; `Dictionary<TKey, TValue>` for fast lookup by key; `HashSet<T>` for uniqueness checks; `Queue<T>` for FIFO processing; `Stack<T>` for LIFO — pick based on how the data will actually be accessed, not habit.

Structured exception handling. `try { ... } catch (SpecificException ex) { ... } finally { ... }` — the `finally` block always runs, whether or not an exception occurred, making it the right place for cleanup that must always happen.

Custom exceptions. `public class InvalidGradeException : Exception` lets you throw and catch a domain-specific error type, carrying more meaning than a generic exception and letting calling code catch exactly the failure it knows how to handle.

Delegates and lambda expressions. A delegate is a type-safe reference to a method; a lambda `(s => s.Average > 70)` is a concise, inline way to write the small function LINQ methods like `Where` and `Select` expect as an argument.

Nullable types and null-handling operators. `?.` safely accesses a member only if the object isn't null; `??` supplies a fallback when the left side is null; `??=` assigns only if the variable is currently null.

Practical example

LINQ querying, custom exceptions, and null-safe access together

```

public class InvalidGradeException : Exception
{
    public InvalidGradeException(string message) : base(message) { }
}

void RecordGrade(Student s, double grade)
{
    if (grade < 0 || grade > 100)
        throw new InvalidGradeException($"{grade} is not a valid grade");
    s.Grades.Add(grade);
}

var topThree = students
    .Where(s => s.Grades.Any())
    .OrderByDescending(s => s.Grades.Average())
    .Take(3)
    .Select(s => s.Name)
    .ToList(); // executes here

string firstName = topThree.FirstOrDefault() ?? "No students yet";

```

Walking through it: InvalidGradeException is a custom exception type carrying a specific meaning; RecordGrade throws it when given an out-of-range value, which calling code can catch specifically with catch (InvalidGradeException ex). The LINQ chain reads almost like a sentence — keep students with grades, sort by average descending, take the top three, pull out just their names — and doesn't actually run until .ToList() forces enumeration. FirstOrDefault() returns null if the list is empty, and ?? supplies a fallback string instead of crashing on a null reference.

Lesson 15: Introduction to .NET & ASP.NET Core

What it is. ASP.NET Core is Microsoft's modern, cross-platform framework for building web applications and RESTful APIs on top of .NET. This lesson is about the shape of a project — how it starts, how it's configured, how a request actually flows through it — before writing real endpoints in it.

Why it matters. This lesson establishes the foundation for every RESTful API built for the rest of the module, and introduces the exact architecture used in enterprise .NET applications across South African corporates that build on Microsoft technology.

How it works

The .NET ecosystem. ".NET" is the underlying runtime and platform; ASP.NET Core is the web framework built on top of it. Modern .NET (5 and later) unified the older, Windows-only .NET Framework and the cross-platform .NET Core into one platform.

MVC vs Web API. MVC returns server-rendered HTML views; a Web API returns data — typically JSON — with no views at all, meant to be consumed by a separate frontend like React. This module focuses on the Web API side.

Installing and running the .NET SDK. The .NET CLI (`dotnet new webapi`, `dotnet run`) scaffolds and runs a project from the terminal; Visual Studio wraps the same tooling with a GUI.

Project structure. `Program.cs` is the entry point in the modern minimal-hosting model, where you register services and build the middleware pipeline in one place; `appsettings.json` holds configuration like connection strings.

Dependency Injection. Services are registered with a lifetime — `AddTransient` (new instance every time it's requested), `AddScoped` (one instance per HTTP request), `AddSingleton` (one instance for the whole app's lifetime) — and the built-in container hands the right instance to any constructor that asks for it.

Middleware and the request pipeline. Each `app.Use...` call in `Program.cs` adds a stage the request passes through in order — logging, authentication, routing — each able to inspect, modify, or short-circuit the request before passing it along.

Creating and testing a basic Web API. `app.MapGet("/ping", () => "pong")` is the simplest possible endpoint; Swagger (covered in a later lesson) gives you an interactive way to try real endpoints from the browser as you build them.

Practical example

A minimal ASP.NET Core API with a registered service

```
var builder = WebApplication.CreateBuilder(args);

// dependency injection: register a scoped service
builder.Services.AddScoped<IStudentService, StudentService>();

var app = builder.Build();

app.Use(async (context, next) =>
{
    Console.WriteLine($"{context.Request.Method} {context.Request.Path}");
    await next(); // pass the request to the next stage of the pipeline
});

app.MapGet("/students", (IStudentService svc) => svc.GetAll());

app.Run();
```

Walking through it: `builder.Services.AddScoped` registers `IStudentService` with the DI container, so any endpoint that asks for it — like the `MapGet` handler below — automatically receives an instance, one per request. The `app.Use` middleware logs every incoming request's method and path, then calls `next()` to pass control to whatever comes after it in the pipeline, which eventually reaches the matching route handler.

Lesson 16: ASP.NET Core & Web API Development

What it is. This lesson turns REST principles into actual working controllers: designing endpoints, routing requests to the right method, handling input, and returning the HTTP status code that accurately describes what happened.

Why it matters. Most real-world React job positions and virtually every professional backend role require solid Web API skills — e-commerce sites, dashboards, content platforms all need a properly structured API behind them.

How it works

REST architecture and HTTP methods. GET reads, POST creates, PUT updates (replacing the whole resource), DELETE removes — the URL names the resource (`/students`), the verb names the action, keeping the API predictable.

Creating controllers and endpoints. `[ApiController]` on a class enables automatic model validation and sensible defaults; each action method handles one endpoint.

Attribute and endpoint routing. `[Route("api/[controller]")]` on the class plus `[HttpGet("{id}")]`/`[HttpPost]` on each action maps an HTTP verb and URL pattern directly to that method.

Implementing CRUD with Web APIs. One controller typically exposes GET (list), GET/{id} (single), POST (create), PUT/{id} (update), and DELETE/{id} (remove) — the same resource, five actions.

Working with request and response objects. A parameter can be bound from the route (`{id}`), the query string, or the body (`[FromBody]` for a complex object) — ASP.NET Core figures out which by default, but a complex type from the body needs `[FromBody]` made explicit.

Returning appropriate status codes. `Ok(data)` (200), `CreatedAtAction(...)` (201, and points the client at the new resource), `NotFound()` (404), `BadRequest(...)` (400) — returned as `ActionResult` or `ActionResult<T>`.

Testing endpoints with Postman. Sending real GET/POST/PUT/DELETE requests with chosen headers and bodies against the running API verifies each endpoint actually behaves as designed, independent of any frontend.

Practical example

A Students controller with full CRUD and correct status codes

```

[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly IStudentService _service;
    public StudentsController(IStudentService service) => _service = service;

    [HttpGet("{id}")]
    public ActionResult<Student> GetById(int id)
    {
        var student = _service.Find(id);
        return student is null ? NotFound() : Ok(student);
    }

    [HttpPost]
    public ActionResult<Student> Create([FromBody] Student s)
    {
        _service.Add(s);
        return CreatedAtAction(nameof(GetById), new { id = s.Id }, s);
    }
}

```

Walking through it: The service is injected through the constructor (dependency injection from the previous lesson in action). `GetById` returns `NotFound()` when the lookup fails and `Ok(student)` otherwise — the status code always matches reality. `Create` binds the request body into a `Student` via `[FromBody]`, adds it through the service, then returns 201 via `CreatedAtAction`, which also tells the client exactly where to GET the new resource from.

Lesson 17: Entity Framework Core & Database Integration

What it is. Entity Framework Core is an ORM (Object-Relational Mapper): it maps C# classes to database tables and lets you query and persist data as ordinary objects, instead of hand-writing SQL for every operation.

Why it matters. This lesson connects the Web API layer to real, persistent data — mirroring the workflow used in professional software engineering environments, where almost nothing important is only ever held in memory.

How it works

Object-Relational Mapping (ORM) concepts. Instead of writing SQL by hand for every operation, you work with C# objects, and EF Core translates operations on them into the equivalent SQL behind the scenes — reducing boilerplate while still ultimately hitting a real relational database.

Configuring EF Core. Registering a `DbContext` in DI

(`builder.Services.AddDbContext<SchoolContext>(...)`) with a connection string from `appsettings.json` is what wires the app up to a specific database provider (SQL Server, in this module).

Entity models and `DbContext`. An entity is a plain C# class representing a table row; `DbContext` represents a session with the database, and each `DbSet<Student>` property on it maps that entity to its table.

Connecting to SQL Server. The connection string in configuration tells EF Core which server and database to talk to; the `DbContext` uses it to open connections as needed when you query or save.

Migrations. `dotnet ef migrations add InitialCreate` compares your current model against the last known schema and generates a script describing the difference; `dotnet ef database update` actually applies it to the database.

CRUD operations with EF Core. `context.Students.Add(s)` stages an insert, `.Find(id)` reads by primary key, `.Remove(s)` stages a delete — none of it reaches the database until `context.SaveChanges()` is called.

Data annotations and Fluent API. Attributes like `[Key]` and `[Required]` configure a model directly on the class; the Fluent API inside `OnModelCreating` handles configuration that doesn't fit neatly as an attribute, like composite keys or relationship details.

Repository and service layer practices. Keeping direct `DbContext` usage inside a service (or repository) layer, rather than scattered through controllers, keeps data-access logic in one place and controllers focused purely on handling HTTP concerns.

Practical example

A `DbContext`, entity model, and full CRUD flow

```

public class Student
{
    public int Id { get; set; }
    [Required]
    public string Name { get; set; }
}

public class SchoolContext : DbContext
{
    public SchoolContext(DbContextOptions<SchoolContext> options) : base(options) { }
    public DbSet<Student> Students => Set<Student>();
}

public class StudentService : IStudentService
{
    private readonly SchoolContext _context;
    public StudentService(SchoolContext context) => _context = context;

    public void Add(Student s)
    {
        _context.Students.Add(s);
        _context.SaveChanges(); // nothing is persisted before this line
    }
}

```

Walking through it: Student is a plain entity class with a [Required] annotation on Name. SchoolContext exposes it as a DbSet, which EF Core maps to a Students table (created by a migration). StudentService receives the context through DI and wraps the actual database interaction — Add() stages the new row in the change tracker, and SaveChanges() is what actually sends the INSERT to the database; skip that call and the student is never really saved.

Lesson 18: API Authentication & Security with JWT

What it is. Authentication and authorization secure an API using JSON Web Tokens: verifying who a user is at login, then trusting a signed, stateless token to carry that identity across every request that follows, without the server needing to remember a session.

Why it matters. Security is a critical, non-optional part of backend development, and this lesson gives practical experience with the exact authentication mechanism — JWT — used across real enterprise applications.

How it works

Authentication vs authorization. Authentication answers 'who are you' — a login endpoint checking credentials. Authorization is the separate, later question of 'what are you allowed to do', checked on each protected request.

Configuring JWT authentication. ASP.NET Core is configured with `AddAuthentication().AddJwtBearer(...)`, telling it how to validate an incoming token's signature, issuer, audience, and expiration on every request.

Generating and validating tokens. On successful login, the server builds a token containing claims (user id, roles, expiry) and signs it with a secret key; on every later request, the framework verifies that signature to trust the claims without needing to look anything up in a session store.

Protecting endpoints with authorization attributes. `[Authorize]` on a controller or action rejects any request without a valid token; omitting it leaves the endpoint public.

Role-based access control. `[Authorize(Roles = "Admin")]` additionally requires the token's claims to include that role, letting different user types reach different endpoints.

Securing sensitive data. Secrets (signing keys, connection strings) belong in configuration or a secrets manager, never hard-coded or committed to source control — the same discipline applies to anything a token or database connection depends on.

Token expiration and refresh. A short expiration limits how long a leaked token stays dangerous; a longer-lived refresh token lets the client silently obtain a new access token without forcing the user to log in again every few minutes.

Practical example

A login endpoint issuing a JWT, and a protected endpoint consuming it

```

[HttpPost("login")]
public IActionResult Login(LoginRequest req)
{
    var user = _users.Validate(req.Username, req.Password);
    if (user is null) return Unauthorized();

    var claims = new[] {
        new Claim(ClaimTypes.Name, user.Username),
        new Claim(ClaimTypes.Role, user.Role)
    };
    var token = new JwtSecurityToken(
        claims: claims,
        expires: DateTime.UtcNow.AddHours(1),
        signingCredentials: _signingCredentials);

    return Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token) });
}

[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult Delete(int id) { _service.Remove(id); return NoContent(); }

```

Walking through it: Login validates the username/password, and only on success builds a token carrying the username and role as claims, signed and set to expire in an hour. The client stores that token and sends it as an Authorization: Bearer header on later requests. Delete is decorated with [Authorize(Roles = "Admin")], so ASP.NET Core checks the incoming token's role claim before the method body ever runs, rejecting anyone without the Admin role automatically.

Lesson 19: API Testing, Validation & Documentation with Swagger

What it is. This lesson closes the ASP.NET Core module by making an API trustworthy and usable by someone else: validating input properly, documenting every endpoint automatically with Swagger, and testing it thoroughly before calling it done.

Why it matters. Proper testing and documentation are essential for delivering reliable, maintainable backend services — and a fully documented, well-tested API is exactly what students showcase on their GitHub portfolios for employers.

How it works

Configuring Swagger/OpenAPI. Adding the Swagger package and calling `app.UseSwagger();` `app.UseSwaggerUI();` generates interactive, browsable documentation directly from your controllers and models — no separate docs to keep in sync by hand.

Model validation with data annotations. `[Required]`, `[StringLength(100)]`, `[Range(0, 100)]` on a request model describe the rules a valid request must satisfy.

Handling validation and exception responses. With `[ApiController]` on the controller, an invalid model automatically produces a 400 response with the specific validation errors before your action method even runs — no manual `ModelState.IsValid` check required (though it's still available for custom logic).

Testing endpoints with Swagger UI and Postman. Swagger UI is fast for exploratory testing straight from the browser while developing; Postman is better for saved, repeatable request collections you can re-run as you keep building.

Understanding OpenAPI specifications. OpenAPI is the underlying specification format Swagger UI renders — a machine-readable description of every endpoint, its parameters, and its responses, which other tools (and other developers) can consume directly.

Improving API usability through documentation. Clear summaries and example values on each endpoint (via XML doc comments or attributes) make the generated Swagger docs actually useful to someone who didn't write the API.

Server-side validation as a security boundary. Client-side checks are a UX convenience; anyone can bypass the frontend and call the API directly with Postman or a script, so the rules that actually matter must be enforced again on the server.

Practical example

A validated request model that Swagger documents automatically

```

public class CreateStudentRequest
{
    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string Name { get; set; }

    [Range(0, 100)]
    public int Average { get; set; }
}

[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    [HttpPost]
    public IActionResult Create(CreateStudentRequest req)
    {
        // [ApiController] already returned 400 automatically if req was invalid
        _service.Add(req);
        return Ok();
    }
}

```

Walking through it: CreateStudentRequest's annotations describe exactly what a valid request looks like: a name between 2 and 100 characters, an average between 0 and 100. Because the controller has [ApiController], sending a request that violates any of those rules never even reaches the Create method body — ASP.NET Core rejects it with a 400 and a description of what was wrong automatically, and Swagger UI shows these exact constraints to anyone exploring the API.

Lesson 20: SQL Server — Queries, Joins & Database Design

What it is. This lesson is the core relational database vocabulary: writing SQL to filter, sort, join, and summarize data, and designing a schema that avoids storing the same fact in multiple places.

Why it matters. SQL is required for virtually every backend developer position, and South African companies across every industry — banking, retail, insurance, government — rely heavily on SQL databases. It's treated as a non-negotiable skill for professional developers.

How it works

Writing SELECT queries. `SELECT columns FROM table WHERE condition ORDER BY column`, optionally limited with `TOP n` — the basic shape nearly every other query builds on.

JOIN operations. `INNER JOIN` returns only rows matching on both sides; `LEFT JOIN` keeps every row from the left table, filling `NULLs` where there's no match on the right; `RIGHT JOIN` is the mirror image; `FULL OUTER JOIN` keeps everything from both sides.

Aggregate functions. `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` summarize a set of rows into a single value per group.

Grouping and filtering data. `GROUP BY` buckets rows before aggregating; `WHERE` filters individual rows before that grouping happens, while `HAVING` filters the resulting groups afterward, which is the only place an aggregate value can actually be filtered on.

Database normalization. 1NF removes repeating groups within a row, 2NF removes data that only depends on part of a composite key, 3NF removes data that depends on a non-key column rather than the key itself — each level removing a specific kind of redundancy that could otherwise go inconsistent.

Keys and constraints. A primary key uniquely identifies each row; a foreign key constrains a column to only reference a real row in another table, enforcing relational integrity at the database level rather than trusting application code to get it right.

Indexes and performance. An index on a frequently-searched column dramatically speeds up reads on it, at the cost of slightly slower writes (the index has to be updated too) and extra storage — a deliberate trade-off, not a free win.

Subqueries, CTEs, and transactions. A subquery or a Common Table Expression (`WITH ... AS (...)`) lets you build a query in named, readable stages; a transaction (`BEGIN TRAN ... COMMIT`) groups several statements so they all succeed or all roll back together, keeping data consistent.

Practical example

Joining, grouping, and filtering enrollment data

```
SELECT c.Name AS Course, COUNT(e.StudentId) AS Enrolled, AVG(e.Grade) AS AvgGrade
FROM Courses c
LEFT JOIN Enrollments e ON e.CourseId = c.Id
GROUP BY c.Name
HAVING COUNT(e.StudentId) > 5
ORDER BY AvgGrade DESC;
```

Walking through it: The LEFT JOIN keeps every course even if it has zero enrollments, filling in NULLs for e.StudentId/e.Grade on those rows. GROUP BY c.Name buckets all enrollment rows by course, so COUNT and AVG are computed per course rather than across the whole table. HAVING COUNT(e.StudentId) > 5 only keeps courses with more than five enrolled students — written as WHERE, this would fail, since the count doesn't exist until after grouping.

Lesson 21: NoSQL with MongoDB & Deployment

What it is. This closing lesson introduces document databases as an alternative to relational SQL, and covers actually shipping a full-stack app to production: a React frontend to GitHub Pages and an ASP.NET Core API to Azure App Service.

Why it matters. Deployment skills are crucial — students must be able to showcase live, working projects to potential employers, not just code that runs locally. Understanding both SQL and NoSQL also makes a developer more versatile: South African startups often favor MongoDB's flexibility, while enterprises lean on SQL Server.

How it works

SQL vs NoSQL. SQL databases enforce a fixed schema and are strong at relational joins and strict consistency; MongoDB stores flexible-schema, JSON-like documents (BSON), trading some consistency guarantees for schema flexibility and easier horizontal scaling.

MongoDB basics: collections and documents. A collection is roughly like a table, but the documents inside it don't have to share an identical set of fields — each document can have its own shape, which suits data that varies or evolves quickly.

CRUD operations in MongoDB. `db.students.insertOne({...})` creates, `.find({...})` reads with a filter, `.updateOne({...}, { $set: {...} })` updates, and `.deleteOne({...})` removes a document.

Queries with filters, projections, and aggregations. A filter narrows which documents match (`{ average: { $gt: 70 } }`); a projection picks which fields come back; the aggregation pipeline chains multiple transform stages together for more complex reporting-style queries.

Choosing between SQL and NoSQL. Favor SQL when relationships and strict consistency matter most (financial records, enrollments); favor NoSQL when the data's shape varies a lot or needs to scale out horizontally with minimal friction.

Deploying a React frontend. A React app's production build is just static files (`npm run build`), which deploy well to GitHub Pages, including with a custom domain.

Deploying an ASP.NET Core API. Azure App Service's free tier can host a live ASP.NET Core API, configured with environment variables and connection strings kept out of source control rather than hard-coded.

Configuring CORS for production. The deployed API must explicitly allow the deployed frontend's exact origin in its CORS policy, or the browser blocks the frontend's requests — a restriction enforced by the browser itself, not by the server or by tools like Postman.

Practical example

A MongoDB query and enabling CORS for a deployed frontend

```
// MongoDB: students averaging above 70, best first
db.students.find(
  { average: { $gt: 70 } },
  { name: 1, average: 1, _id: 0 } // projection: only these fields
).sort({ average: -1 });

// ASP.NET Core: allow the deployed frontend's exact origin
builder.Services.AddCors(options =>
  options.AddPolicy("Frontend", p =>
    p.WithOrigins("https://myapp.github.io")
      .AllowAnyMethod()
      .AllowAnyHeader()));

app.UseCors("Frontend");
```

Walking through it: The MongoDB query filters for students above a 70 average, and the projection ({ name: 1, average: 1, _id: 0 }) trims the response down to just the fields actually needed. The CORS policy explicitly names the deployed GitHub Pages origin as allowed; without `app.UseCors("Frontend")` wired into the pipeline, a browser running the live site would block every request to the API even though the exact same request works fine from Postman, which never enforces CORS at all.